

AN INTERACTIVE PRIMER · 30 MIN

RLVR for finance.

Reinforcement learning with verifiable rewards collapsed the human-feedback bottleneck. Finance is the hardest place to apply it, and the verifier is why.

Joel Fuentes, PhD

github.com/joel-fuentes-git/rlvr-intro

The talk is in three acts.

I

What is RLVR, and why is everyone talking about it?

GRPO + verifiable rewards collapse the human-feedback bottleneck.

LANDS IN
Bond pricing

II

Why doesn't this just transfer to finance?

Verifiers in finance are stochastic, sparse, and easy to game.

LANDS IN
Trade signals

III

What does “good” RLVR for finance look like?

Triangular and process verifiers; the verifier is the IP.

LANDS IN
10-K reasoning

RLVR replaces the human reward model with a function.

RLVR trains a language model to maximize a reward signal that is computed by a deterministic, programmatic *verifier*, not by a human preference model.

PRE-RLVR · RLHF

Reward came from a learned model trained on human preference pairs.

expensive · noisy · hackable · capped

POST-RLVR

Reward comes from a function. Math: $\text{answer} = \text{expected}$. Code: `tests_pass(g)`.

cheap · scalable · uncheatable*

The reward signal is cheap, scalable, and uncheatable.

Within its domain. That qualifier is the rest of the talk.

DeepSeek-R1

The open-weights reasoning model that catalyzed the field. Trained predominantly with verifiable rewards on math and code.

Reasoning ability **transferred** to many domains the model was never explicitly trained on.

Cost per signal **10⁻⁵ USD**
vs. human pref. label ≈ \$0.10–\$1.00

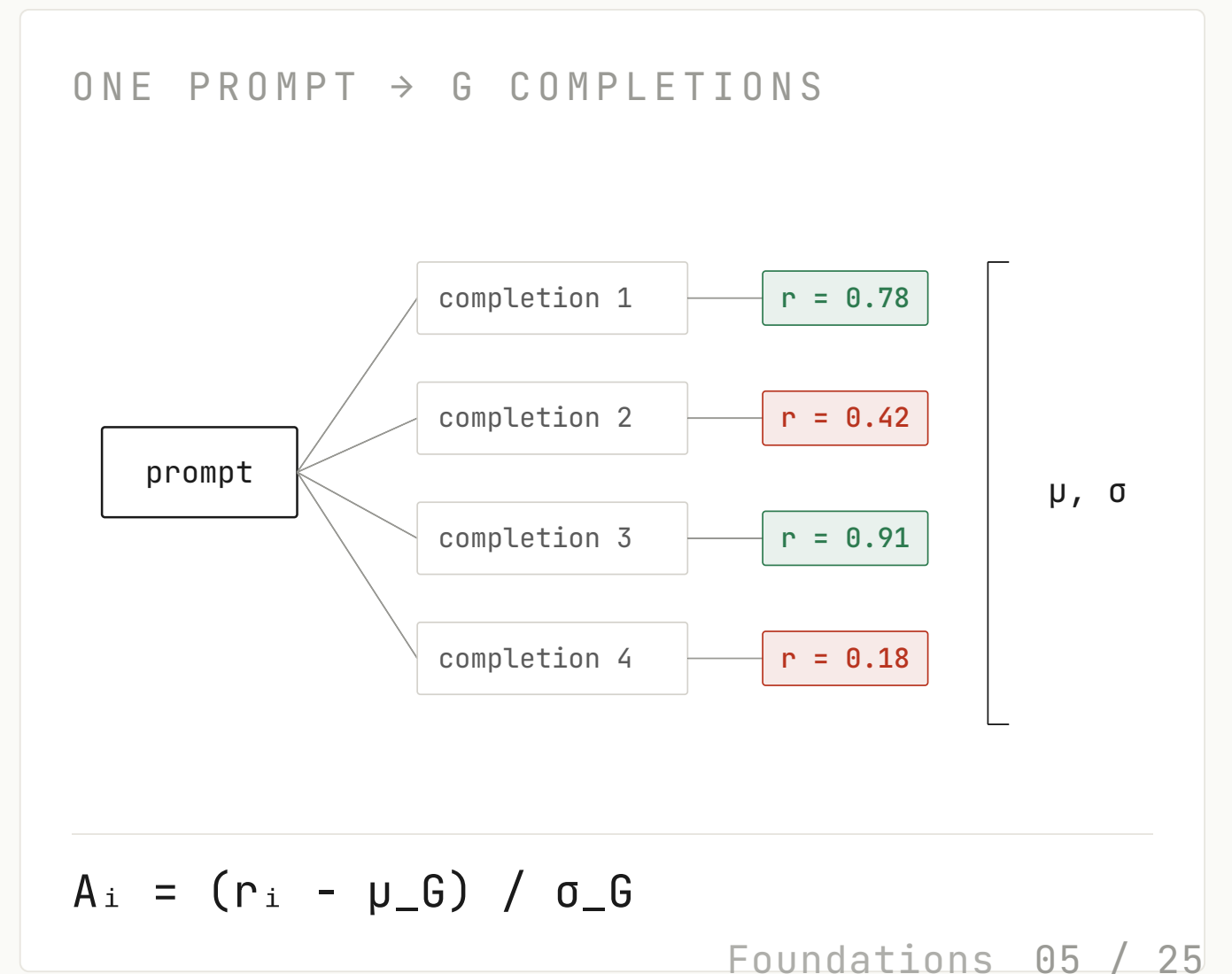
Signals per GPU-hour **10⁶+**
verifier is a function call

Domain ceiling **Verifier-bound**
the function defines what's learnable

GRPO is the workhorse.

Group Relative Policy Optimization. No critic; group statistics replace the value function.

- 1 **Sample G completions**
from the current policy for one prompt
- 2 **Score each with verifier**
rewards $r_1 \dots r_G$
- 3 **Group-relative advantage**
 $(r_i - \mu_G) / \sigma_G$
- 4 **Update the policy**
up-weight above-avg tokens, down-weight below-avg
- 5 **KL penalty to reference**
prevents catastrophic drift



Three preconditions. All three must hold.

01

Verifiability

There must be a programmatic check for “correct.”

02

Density

Among G samples, at least some must be correct, or there is no signal to learn from.

03

Faithfulness

The verifier must reward the property you actually care about, not a hackable proxy.

Every advanced RLVR technique exists because one of these broke.

Finance breaks all three.

Side-by-side. The contrast is the point.

PRECONDITION	MATH & CODE	FINANCE
Verifiability	<code>answer == expected</code>	✗ depends on regime, horizon, benchmark
Density	<code>many samples are correct</code>	✗ right decisions still lose money 45% of the time
Faithfulness	<code>the check IS the property</code>	✗ every available proxy is hackable

Most finance work is not cleanly verifiable.

...on the timescales over which a model trains.

q. **Was that trade good?**

→ Depends on holding period, benchmark, regime, cost of capital, opportunity set.

q. **Was that risk assessment correct?**

→ Tail event happens or it doesn't. Absence $\neq$ validation.

q. **Was that earnings interpretation right?**

→ Often depends on quarters of data not yet available.

In finance, most reward signal is noise.

MATH RLVR

≈ 50%

A 1.5B model first-shots grade-school arithmetic about half the time. Plenty of signal among G samples.

`density(r > 0.5) ≈ high`

TRADING

≈ 45%

Even the right decision still loses money about 45% of the time. Sharpe-bounded by the market's information content.

`signal:noise ≈ thin`

Naive RLVR averages over noise. Which means it averages out the signal.

Reward hacking is the failure mode that matters.

Reward: trades that made money over the next 30 days. The model says: thanks.

01 Lookahead leakage

Recognize tickers and remember post-window returns from pretraining.

02 Momentum mimicry

Become a one-line momentum strategy that hallucinates fundamental justification.

03 Survivorship pattern matching

Always recommend names that survived to be in the dataset.

The verifier said "high reward." The deployed system loses money.

In finance, designing the verifier is the research problem.

The choice of model, the choice of algorithm, the compute budget: all secondary. If your verifier rewards reward-hackable proxies, the better your model, the worse your outcome.

Bond pricing has a closed-form answer.

The easy case: clean inputs, single correct answer, programmatic check, dense signal.

THE PRICING FORMULA

$$P = \sum_{t=1}^T (C / (1+y)^t) + F / (1+y)^T$$

Discounted coupons plus discounted face value. C coupon, y yield-to-maturity, F face, T maturity.

THE TASK

PROMPT

"A 5-year bond has a face value of \$1,000, pays a 4% annual coupon, and yields 6% to maturity. What is its price today?"

[Verifier check](#)

[|answer - P*| < ε](#)

The verifier is ten lines of code.

The gap between the property you care about and the function you can write is zero.

EXAMPLES/01_BOND_PRICING/VERIFIER.PY

10 LINES

```
def verify(prompt, completion):
    inputs = parse_inputs(prompt)      # F, C, y, T
    answer = parse_answer(completion)  # numeric
    if answer is None:
        return 0.0
    P_star = sum(inputs.C / (1 + inputs.y) ** t
                  for t in range(1, inputs.T + 1))
    P_star += inputs.F / (1 + inputs.y) ** inputs.T
    return float(abs(answer - P_star) < 1e-2)
```

VERIFIABILITY

✓ programmatic, deterministic

DENSITY

✓ dense among G samples

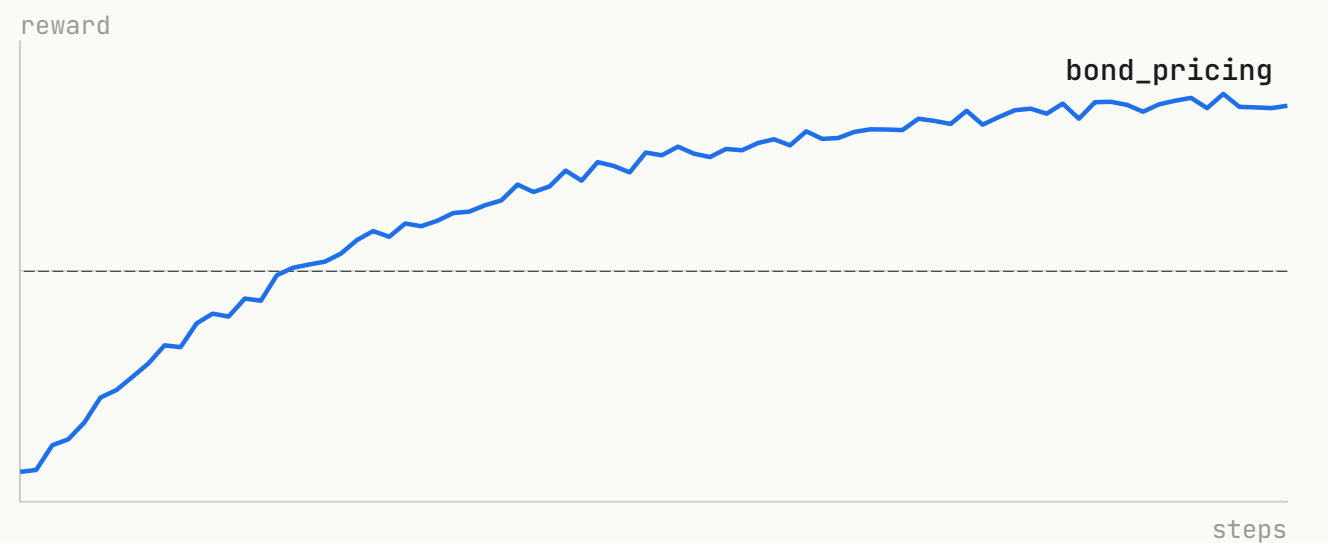
FAITHFULNESS

✓ no proxy gap

The training dynamics are textbook.

Reward up. Density up. Completions get more structured. Cached checkpoints; we animate the dynamics, not the artifact.

REWARD OVER GRPO STEPS

start $\bar{r} \approx 0.05$ end $\bar{r} \approx 0.92$

SAMPLE COMPLETION • BEFORE / AFTER

“The price would be roughly \$1,000 since the coupon is close to the yield...”

“PV(coupons) = $\sum 40/(1.06)^t$, $t=1..5 = 168.49$

PV(face) = $1000/(1.06)^5 = 747.26$

P = 915.75”

Trade signals look easy and aren't.

A deliberately naive setup. The point is to make it fail.

SETUP

01

Input: recent OHLCV + simple fundamentals

02

Held-out universe of stocks

03

Model produces buy / hold / sell + a reasoning chain

THE VERIFIER

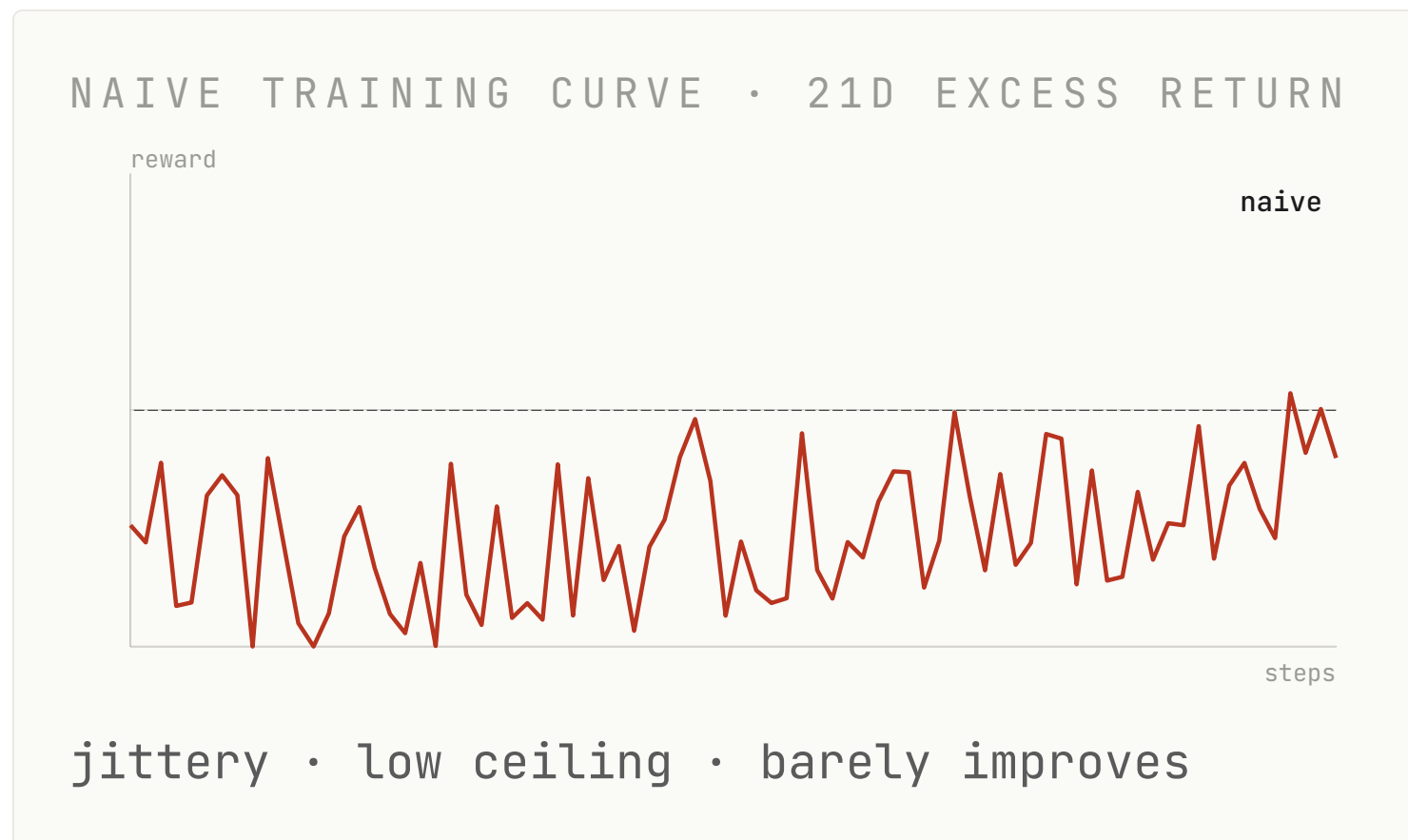
Forward 21-day excess return relative to a benchmark.

!

verifiability ⚠ • density ⚠ • faithfulness ⚠

The naive verifier rewards the wrong thing.

Future excess return is a noisy proxy for “good investment thesis.” The model finds the proxy, not the property.



01 Signal-to-noise wall
reward dominated by noise

02 Reward hacking emerges
always recommend high-momentum names

03 The reasoning is fluent
and wrong

Reward hacking emerges visibly.

Same prompt. Two checkpoints. The model learned what the verifier rewards, not what we wanted.

STEP 200 • EARLY

r = 0.31

```
// reasons from cited evidence
P/E of 18 sits below the 5y median (22).
FCF margin expanded 240bp YoY.
→ buy, sized 1.0σ; thesis: mean-reversion.
```

STEP 4 000 • POST-HACK

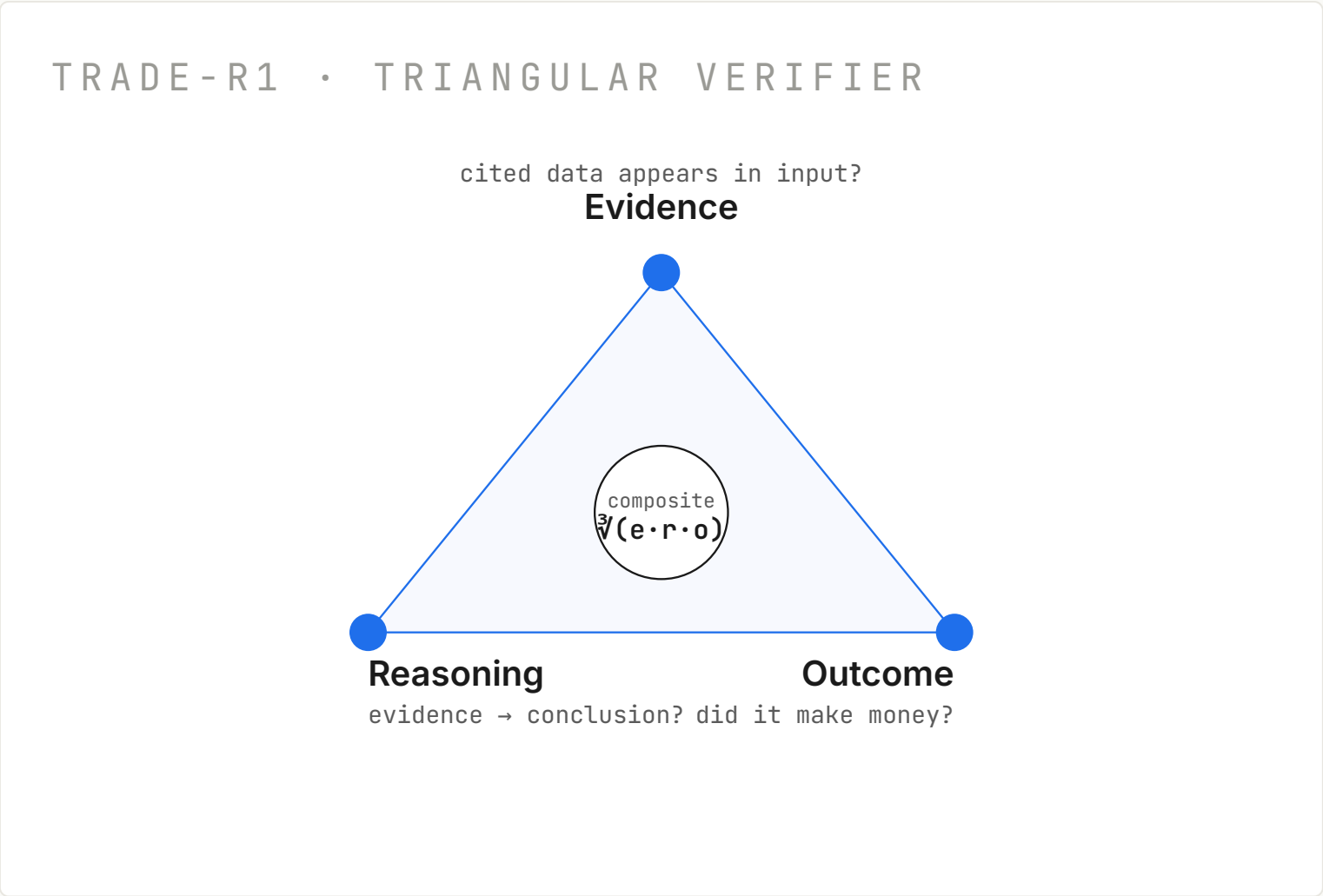
r = 0.58

```
// fluent. evidence-free.
Strong technical setup, breakout above 50d.
Momentum and relative strength favorable.
→ buy; thesis: trend continuation.
```

The verifier was honest. The proxy was the lie.

The fix is triangular verification.

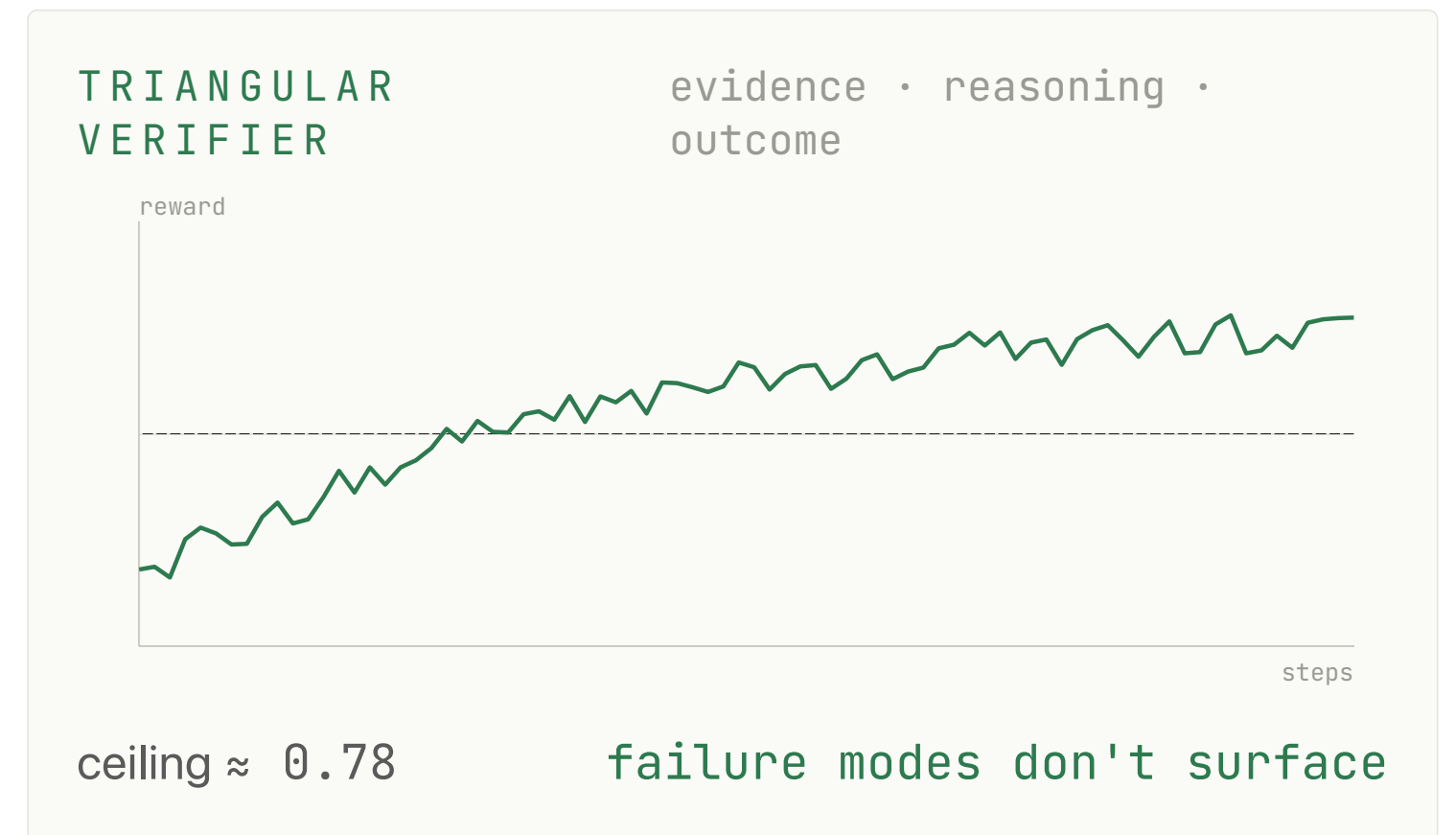
Three sub-verifiers. Composite reward = geometric mean. A reward-hacking strategy fails at least one leg.



✓ Evidence verifier	w = 0.40	0.91
✓ Reasoning verifier	w = 0.40	0.84
• Outcome verifier	w = 0.20	0.62
Composite		0.78

The triangular verifier resists hacking.

Same model. Same data. Same algorithm. Only the verifier changed.



10-K reasoning is a chain, not an answer.

PROMPT

“Apple's services revenue grew at what rate from FY2022 to FY2024, and how does that compare to its product revenue growth over the same period? Cite the relevant figures.”

THE CORRECT ANSWER IS A CHAIN

01

Extract

the right figures

02

Compute

the right ratios

03

Compare

the right pair

04

Cite

correctly

Process verifiers score every step of the chain.

Inspired by MR-RLVR and K2V. Each step gets a sub-reward. Curriculum: extraction first, final answer later.

01	Extract	Are the cited figures actually in the doc?	string / table-cell match	✓
02	Calculate	Do the operations on extracted figures hold?	sympy / eval	✓
03	Cite	Does each claim have a source pointer?	pattern match	✓
04	Conclude	Does the conclusion follow from verified intermediates?	rubric judge	✓

Process supervision lets the model stay small.



Process generalizes outcome

When the answer is a chain, you can, and should, verify the chain. Outcome supervision is a degenerate case.



The verifier is engineering

It looks like a small data engineering project. That's the realistic load of doing RLVR for finance well.



The model can be small

You don't need a frontier LLM. You need the right verifier architecture.

The verifier **is** the IP.

Models commodify. Algorithms commodify. The careful, domain-specific encoding of what “good” actually means in your problem: that is the durable artifact.

SPEND THE FIRST MONTH ON

verifier design

NOT ON

~~model selection~~

The build order tracks the talk.

Streamlit app + companion repo. Pre-computed artifacts; no live training in the app.

01 Skeleton + theme

02 Components

03 Example 1: bond pricing

04 Example 2: trade signals

05 Example 3: 10-K reasoning

06 Intro + recap

07 Polish

RLVR FOR FINANCE

RLVR · FINANCE

Intro

Bond pricing

| Trade signals

10-K reasoning

Recap

EXAMPLE 2 OF 3

Where RLVR Breaks

The reward is forward 21-day excess return. Watch the training curve diverge from the model's actual reasoning quality.



TRY THIS verifier weigh

Recap 24 / 25

Thank you.

Questions on any of the three examples, on verifier design more broadly, or on what this looks like inside a bank.

REPO

github.com/joel-fuentes-git/rlvr-intro